

HashMap Level 3 Problems

1. Longest Consecutive Sequence → HashSet + Sequence

```
nums = [100,4,200,1,3,2]
# nums = [0,3,7,2,5,8,4,6,0,1]
def longest_consec(nums):

    nums_set = set(nums)
    longest = 0

    for num in nums_set:

        if num - 1 not in nums_set:
            current = num
            length = 1

            while current + 1 in nums_set:
                current += 1
                length += 1

            longest = max(longest,length)

    return longest
print(longest_consec(nums))
```

2. Subarray Sum Equals K → Prefix Sum + HashMap

```
nums = [1, 2, 3]
# nums = [7, 1, 1,10]
k = 3

def subarr_sum(nums,k):

    pre_cnt = {0:1}
    pre = 0
    cnt = 0

    for num in nums:
        pre += num
        needed = pre - k

        if needed in pre_cnt:
            cnt += pre_cnt[needed]

    pre_cnt[pre] = pre_cnt.get(pre,0) + 1

    return cnt

print(subarr_sum(nums,k))
```

3. Find All Duplicates → Frequency + HashMap

```
nums = [4,3,2,7,8,2,3,1]

def find_duplicates(nums):

    freq = {}

    for i in nums:
        freq[i] = freq.get(i,0) + 1

    result = []

    for i in freq:
        if freq[i] == 2:
            result.append(i)

    return result

print(find_duplicates(nums))
```

4. Top K Frequent Elements → Frequency + Heap

```
import heapq
nums = [1,1,1,2,2,3]
k = 2
def top_k_freq(nums,k):
    freq = {}

    for num in nums:
        freq[num] = freq.get(num,0) + 1

    heap = []

    for num, freq in freq.items():
        heapq.heappush(heap,(freq,num))

    if len(heap) > k:
        heapq.heappop(heap)

    return [num for freq, num in heap]

print(top_k_freq(nums,k))
```

5. Group Anagrams → HashMap + Grouping

```
strs = ["eat", "tea", "tan", "ate", "nat", "bat"]

def grp_anag(strs):
    grp = {}

    for word in strs:
        key = "".join(sorted(word))

        if key not in grp:
            grp[key] = []

        grp[key].append(word)

    return list(grp.values())

print(grp_anag(strs))
```

6. Find All Anagrams → HashMap + Sliding Window

```
s = "cbaebabacd"
p = "abc"

def find_anagrams(s, p):

    result = []

    p_freq = {}

    for ch in p:
        p_freq[ch] = p_freq.get(ch, 0) + 1

    window_freq = {}

    left = 0

    for right in range(len(s)):

        window_freq[s[right]] = window_freq.get(s[right], 0) + 1

        if right - left + 1 > len(p):

            old = s[left]

            window_freq[old] -= 1

            if window_freq[old] == 0:
                del window_freq[old]

            left += 1

        if window_freq == p_freq:
            result.append(left)

    return result
```

7. Longest Substring Without Repeating → HashMap/Set + Sliding Window

```
s = "abcabcbb"

def long_substr(s):
    seen = set()
    left = 0
    longest = 0
    for right in range(len(s)):

        while s[right] in seen:
            seen.remove(s[left])
            left += 1

        seen.add(s[right])
        longest = max(longest, right - left + 1)

    return longest
print(long_substr(s))
```

8. Continuous Subarray Sum

→ Prefix Sum + Remainder

```
nums = [23, 2, 4, 6, 7]
k = 6

def check_subarray_sum(nums, k):

    remainder_index = {0: -1}

    prefix = 0

    for i, num in enumerate(nums):

        prefix += num

        remainder = prefix % k

        if remainder in remainder_index:

            if i - remainder_index[remainder] >= 2:
                return True

        else:
            remainder_index[remainder] = i

    return

print(check_subarray_sum(nums,k))
```

9. Contiguous Array

→ 0 → -1 + Prefix Sum

```
nums = [0,1,0]

def find_max_length(nums):

    first_seen = {0: -1}

    prefix = 0
    longest = 0

    for i, num in enumerate(nums):

        if num == 0:
            prefix -= 1
        else:
            prefix += 1

        if prefix in first_seen:

            length = i - first_seen[prefix]

            longest = max(longest, length)

        else:
            first_seen[prefix] = i

    return longest

print(find_max_length(nums))
```

10. Number of Good Pairs

→ Frequency + Pair Counting

```
nums = [1,2,3,1,1,3]

def num_iden_pairs(nums):
    freq = {}

    pairs = 0

    for num in nums:
        if num in freq:
            pairs += freq[num]

        freq[num] = freq.get(num,0) + 1

    return pairs

print(num_iden_pairs(nums))
```